

WAP in C to find second smallest element in an array:-

Program Code

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter number of elements in an array: ");
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    printf("Enter elements: ");
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    }
```

```
    if (n < 2) {
```

```
        printf("array must have atleast two elements.\n");
```

```
        return 0;
```

```
    } int sml, ssm;
```

```
    if (arr[0] < arr[1]) {
```

```
        sml = arr[0];
```

```
        ssm = arr[1];
```

```
    } else {
```

```
        sml = arr[1];
```

```
        ssm = arr[0];
```

```
    } for (int i = 2; i < n; i++) {
```

```
        if (arr[i] < sml) {
```

```
            sml = arr[i];
```

```
            ssm = arr[i];
```

```
        } else if (arr[i] < ssm && arr[i] != sml) {
```

```
            ssm = arr[i];
```

```
        } printf("Average: %d", (sml + ssm) / 2);
```

```
        printf("Average: %d", (sml + ssm) / 2);
```

```
    } printf("Second smallest element is %d\n", sml);
```

```
}
```

Output Code

Enter number of elements in an array: 4

Enter elements: 2 4 5 6

Second smallest element is 4

Program 2

2

to find the sum of the left diagonals of a matrix.

Program Code

#include <stdio.h>

int main() {

int n;

printf("For size of square matrix $n \times n$, Enter n: ");

scanf("%d", &n);

int mat[n][n];

for (int i = 0; i < n; i++) {

for (int j = 0; j < n; j++) {

printf("Enter element matrix [%d][%d] : ", i, j);

scanf("%d", &mat[i][j]);

}

int sum = 0;

for (int i = 0; i < n; i++) {

for (int j = 0; j < n; j++) {

if (i == j) sum = sum + mat[i][j];

}

printf("Left diagonal sum : %d\n", sum);

}

Output of Code

For size of square matrix $n \times n$, Enter n : 2

Enter element matrix [0][0] : 1

Enter element matrix [0][1] : 0

Enter element matrix [1][0] : 0

Enter element matrix [1][1] : 1

Left diagonal sum : 2

Program 3

WAP to find the sum of rows and columns of a matrix.

Program Code:-

```
#include <stdio.h>
```

```
int main () {
```

```
    int m, n;
```

```
    printf ("For size of matrix mxn, Enter m, n : ");
```

```
    scanf ("%d %d", &m, &n);
```

```
    int mat[m][n];
```

```
    for (int i = 0; i < m; i++) {
```

```
        for (int j = 0; j < n; j++) {
```

```
            printf ("Enter element matrix [%d][%d] : ", i+1, j+1);
```

```
            scanf ("%d", &mat[i][j]);
```

```
        }
```

```
    } for (int i = 0; i < m; i++) {
```

```
        int Rsum = 0;
```

```
        for (int j = 0; j < n; j++) {
```

```
            Rsum = Rsum + mat[i][j];
```

```
        }
```

```
        printf ("Row %d sum : %d\n", i+1, Rsum);
```

```
    }
```

```
    for (int i = 0; i < n; i++) {
```

```
        int Csum = 0;
```

```
        for (int j = 0; j < m; j++) {
```

```
            Csum += mat[j][i];
```

```
        }
```

```
        printf ("Column %d sum : %d\n", i+1, Csum);
```

```
    }
```

```
}
```

Output

For size of matrix mxn, Enter m, n : 2 3

Enter element matrix [1][1] : 2

Enter element matrix [1][2] : 3

Enter element matrix [1][3] : 4

Enter element matrix [2][1] : 5

Enter element matrix [2][2] : 6

Enter element matrix [2][3] : 3

Row 1 sum : 9

Row 2 sum : 14

Column 1 sum : 7

Column 2 sum : 9

Column 3 sum : 7

Program 4
MAP to count the total number of duplicate element in an array.

Program Code

```
#include <stdio.h>
int main(1){
    int n;
    printf("Enter the number of elements in an array: ");
    scanf("%d", &n);
    int d[n];
    for(int i=0; i<n; i++){
        scanf("%d", &a[i]);
    }
    int c=0;
    int v[n];
    for(int i=0; i<n; i++) v[i]=0;
    for(int i=0; i<n; i++){
        if(v[i]==1) continue;
        int d=0;
        for(int j=i+1; j<n; j++){
            if(a[i]==a[j]){
                d=1;
                v[j]=1;
            }
        }
        if(d) c++;
    }
    printf("Total number of duplicate elements: %d\n", c);
    return 0;
}
```

Output

Enter number of elements in an array: 7

1 2 3 2 6 1 7

Total number of duplicate elements: 2

Program 5

WAP to find second largest element in an array.

Program Code

```
#include <stdio.h>
```

```
int main() {
```

```
    int n;
```

```
    printf("Enter number of elements in an array: ");
```

```
    scanf("%d", &n);
```

```
    int arr[n];
```

```
    printf("Enter elements: ");
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf("%d", &arr[i]);
```

```
    } if (n < 2) {
```

```
        printf("Array must have atleast two elements.\n");
```

```
        return 0;
```

```
    } int lar, slar;
```

```
    if (arr[0] > arr[1]) {
```

```
        lar = arr[0];
```

```
        slar = arr[1];
```

```
    } else {
```

```
        lar = arr[1];
```

```
        slar = arr[0];
```

```
    for (int i = 2; i < n; i++) {
```

```
        if (arr[i] > lar) {
```

```
            slar = lar;
```

```
            lar = arr[i];
```

```
        } else if (arr[i] > slar && arr[i] != lar) {
```

```
            slar = arr[i];
```

```
        } } printf("Second largest element is %d\n", slar);
```

```
    return 0;
```

```
}
```

Output

Enter number of elements in an array: 5

Enter elements: 2 3 4 5 6 1

Second largest element is 5

Program 6

(6)

WAP to delete an element at a desired position from an array

Program Code

```
#include <stdio.h>
```

```
int main () {
```

```
    int n;  
    printf ("Enter number of elements in an array:");
```

```
    scanf ("%d", &n);
```

```
    int arr[n];
```

```
    printf ("Enter element:");
```

```
    for (int i = 0; i < n; i++) {
```

```
        scanf ("%d", &arr[i]);
```

```
    }  
    int pos;
```

```
    printf ("Enter the position to delete (1 to %d):", n);
```

```
    scanf ("%d", &pos);
```

```
    if (pos < 1 || pos > n) {
```

```
        printf ("Invalid position!\n");
```

```
        return 0;
```

```
    }
```

```
    for (int i = pos - 1; i < n - 1; i++) {
```

```
        arr[i] = arr[i + 1];
```

```
    }  
    n--;
```

```
    printf ("Array after deletion:");
```

```
    for (int i = 0; i < n; i++) {
```

```
        printf ("%d", arr[i]);
```

```
    }  
    printf ("\n");
```

```
    return 0;
```

Output

Enter number of elements in an array: 6

Enter ~~number~~ elements: 4 5 1 2 7 8

Enter the position to delete (1 to 6): 3

Array, after deletion: 4 5 2 7 8

Def 9/3/26

Program-1 (FCFS)

Program code

```

#include <stdio.h>
int main() {
    int n, avg-wt = 0, avg-tat = 0, avg-rt = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    int AT[n], BT[n], CT[n], TAT[n], WT[n], RT[n], RQ[n];
    for (int i = 0; i < n; i++) {
        RQ[i] = 1;
        printf("Enter the arrival time & burst time for P%d: ", i+1);
        scanf("%d %d", &AT[i], &BT[i]);
    }
    for (int i = 0; i < n-1; i++) {
        for (int j = i+1; j < n; j++) {
            if (AT[RQ[i]] > AT[RQ[j]]) {
                int temp = RQ[i];
                RQ[i] = RQ[j];
                RQ[j] = temp;
                avg-wt += AT
            }
        }
    }
    int t = 0;
    for (int i = 0; i < n; i++) {
        if (t < AT[RQ[i]]) {
            t = AT[RQ[i]];
        }
        CT[RQ[i]] = t + BT[RQ[i]];
        TAT[RQ[i]] = CT[RQ[i]] - AT[RQ[i]];
        WT[RQ[i]] = TAT[RQ[i]] - BT[RQ[i]];
        RT[RQ[i]] = t - AT[RQ[i]];
        t = CT[RQ[i]];
    }
    printf("\n FCFS process\n");
    printf("Process\tAT\tBT\tCT\tTAT\tWT\tRT\n");
    for (int i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            i+1, AT[i], BT[i], CT[i], TAT[i], WT[i], RT[i]);
    }
}

```

Output

Enter number of processes : 4

Enter arrival time & burst time for P1: 5 4

Enter arrival time & burst time for P2: 0 3

Enter arrival time & burst time for P3: 1 5

Enter arrival time & burst time for P4: 2 8

FCFS proceen

Proceen	AT	BT	CT	TAT	WT	RT
P1	5	4	20	15	11	11
P2	0	3	3	3	0	0
P3	1	5	8	7	2	2
P4	2	8	16	14	6	6

Average TAT = 9.75

Average WT = 4.75

Average RT = 4.75

Program Code

```

#include <stdio.h>
void non-preemptive (int n, int AT[], int BT[]) {
    int CT[n], TAT[n], WT[n], RT[n], completed[n];
    for (int i=0; i<n; i++) {
        completed[i] = 0;
    }
    int t=0, count = 0;
    while (count < n) {
        int in = -1;
        int min_bt = 9999;
        for (int i=0; i<n; i++) {
            if (AT[i] <= t && completed[i] == 0) {
                if (BT[i] < min_bt) {
                    min_bt = BT[i];
                    in = i;
                }
            }
        }
        if (in != -1) {
            RT[in] = t - AT[in];
            CT[in] = t + BT[in];
            TAT[in] = CT[in] - AT[in];
            WT[in] = TAT[in] - RT[in];
            completed[in] = 1;
            t = CT[in];
            count++;
        }
        t++;
    }
    printf("\n SJF Non-Preemptive Result : \n");
    printf("Proces | AT | BT | CT | TAT | WT | RT | \n");
    for (int i=0; i<n; i++) {
        printf("P%d | %d | %d | %d | %d | %d | %d | \n", i+1, AT[i], BT[i], CT[i], TAT[i], WT[i], RT[i]);
    }
}

```

(10)

```

for (int i=0; i<n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, AT[i], BT[i], CT[i], TAT[i], WT[i], RT[i]);
}

```

```

}
void preemptive (int n, int AT[], int BT[]) {
    int CT[n], TAT[n], WT[n], RT[n], temp_BT[n], completed[n];
    int first_time[n];
    for (int i=0; i<n; i++) {
        temp_BT[i] = BT[i];
        completed[i] = 0;
        first_time[i] = -1;
    }
    int t=0, count=0;
    while (count < n) {
        int in = -1;
        int min_bt = 9999;
        for (int i=0; i<n; i++) {
            if (AT[i] <= t && completed[i] == 0) {
                if (temp_BT[i] < min_bt) {
                    min_bt = temp_BT[i];
                    in = i;
                }
                if (temp_BT[i] == min_bt) {
                    if (AT[i] < AT[in]) in = i;
                }
            }
        }
        if (in != -1) {
            if (first_time[in] == -1) {
                first_time[in] = t;
            }
            temp_BT[in]--;
            t++;
            if (temp_BT[in] == 0) {
                CT[in] = t;
                TAT[in] = CT[in] - AT[in];
                WT[in] = TAT[in] - BT[in];
                RT[in] = first_time[in] - AT[in];
                completed[in] = 1;
                count++;
            }
        }
    }
}

```

```
    } else {  
        t++;  
    }  
    printf("\n SJF Preemptive (SRTF) Result: %i", t);  
    printf("Process | AT | BT | CT | ETAT | WT | RT | \n");  
    for (int i = 0; i < n; i++) {  
        printf("P%d | %d | %d | %d | %d | %d | %d | \n",  
            i+1, AT[i], BT[i], CT[i], ETAT[i], WT[i], RT[i]);  
    }  
}
```

```
int main() {  
    int n, choice;  
    printf("Enter number of processes: ");  
    scanf("%d", &n);  
    int AT[n], BT[n];  
    for (int i = 0; i < n; i++) {  
        printf("Enter arrival time & and burst time for P%d: ", i+1);  
        scanf("%d %d", &AT[i], &BT[i]);  
    }  
    printf("\n 1. SJF Non-Preemptive\n 2. SJF Preemptive (SRTF)\n Enter choice: ");  
    scanf("%d", &choice);  
    if (choice == 1) {  
        non_preemptive(n, AT, BT);  
    }  
    else if (choice == 2) {  
        preemptive(n, AT, BT);  
    }  
    else {  
        printf("Invalid choice!");  
    }  
}
```

768 = TAT
768 = TW
768 = TA

Output 1

(12)

Enter number of processes: 4
Enter arrival time and burst time for P1: 0 4
Enter arrival time and burst time for P2: 1 4
Enter arrival time and burst time for P3: 2 6
Enter arrival time and burst time for P4: 4 8

1. SJF Non-preemptive
2. SJF Preemptive (SRTF)

Enter choice: 1

SJF Non-preemptive Result:

Process	AT	BT	CT	TAT	WT	RT	Average TAT = 8.75 Average WT = 3.75 Average RT = 3.75
P1	0	4	4	4	0	0	
P2	1	4	8	7	3	3	
P3	2	6	14	12	6	6	
P4	4	8	22	18	10	10	

Output 2

Enter number of processes: 4
Enter arrival time and burst time for P1: 0 4
Enter arrival time and burst time for P2: 1 5
Enter arrival time and burst time for P3: 2 7
Enter arrival time and burst time for P4: 4 6

1. SJF Non-preemptive
2. SJF Preemptive (SRTF)

Enter choice: 2

SJF Preemptive (SRTF) Result:

Process	AT	BT	CT	TAT	WT	RT
P1	0	4	4	4	0	0
P2	1	5	9	8	3	3
P3	2	7	22	20	13	13
P4	4	6	15	11	5	5

09/03/26

Average TAT = 8.75
Average WT = 3.75
Average RT = 3.75

Comparison

SJF scheduling is more efficient than FCFS because as we can see Average TAT, Wt, RT. is less in SJF compared to FCFS and also in general SJF (preemptive) is more efficient than non-preemptive because it selects the smallest burst time by interruptions.

Efficiency $\Rightarrow$ ~~SJF~~ preemptive $>$ SJF Non-preemptive $>$ FCFS

[Faint handwritten notes and diagrams follow, including queue representations and scheduling logic]

Lab-3

Write a C program to simulate scheduling Algorithm to find TAT, WT, priority (preemptive & non-preemptive).

Program Code

```
#include <stdio.h>
```

```
int main () {
```

```
    int n, i; time = 0, completed = 0, choice;
```

```
    float avg_tat, avg_wt, avg_rt = 0;
```

```
    int at[10], bt[10], pr[0], rt[10];
```

```
    int ct[10], tat[10], wt[10], res[10];
```

```
    int start[10] = {0};
```

```
    int visited[10] = {0};
```

```
    printf ("Enter number of processes: ");
```

```
    scanf ("%d", &n);
```

```
    for (i=0; i<n; i++) {
```

```
        printf ("Enter AT BT priority for P%d: ", i+1);
```

```
        scanf ("%d %d %d", &at[i], &bt[i], &pr[i]);
```

```
        rt[i] = bt[i];
```

```
        start[i] = -1;
```

```
    }
```

```
    printf ("\n 1. Priority Non-Preemptive\n 2. Priority Preemptive\n");
```

```
    printf ("Enter choice: ");
```

```
    scanf ("%d", &choice);
```

```
    printf ("\nantt Chart: \n");
```

```
    if (choice == 1) {
```

```
        while (completed < n) {
```

```
            int idx = -1;
```

```
            int high = 999;
```

```
            for (i=0; i<n; i++) {
```

```
                if (at[i] <= time && visited[i] == 0) {
```

```
                    if (pr[i] < high) {
```

```
                        high = pr[i];
```

```
                        idx = i;
```

```
                }
```

```
            }
```

```
        }
```

```
{ if (idx != -1) {  
    start[idx] = time;  
    res[idx] = start[idx] - at[idx];  
    printf("P%d | ", idx+1);  
    time += bt[idx];  
    ct[idx] = time;  
    tat[idx] = ct[idx] + at[idx];  
    wt[idx] = tat[idx] - bt[idx];  
    visited[idx] = 1;  
    completed++;  
}  
else  
    time++;  
}
```

```
{ else {  
    while (completed < n) {  
        int idx = -1;  
        int high = 999;  
        for (i = 0; i < n; i++) {  
            if (at[i] <= time && rt[i] > 0) {  
                if (pr[i] < high) {  
                    high = pr[i];  
                    idx = i;  
                }  
            }  
        }  
    }  
}
```

```
if (idx != -1) {  
    if (start[idx] == -1) {  
        start[idx] = time;  
        res[idx] = start[idx] - at[idx];  
        printf("P%d | ", idx+1);  
        rt[idx]--;  
        time++;  
        if (rt[idx] == 0) {
```

```

        ct[idx] = time;
        tat[idx] = ct[idx] - at[idx];
        wt[idx] = tat[idx] - bt[idx];
        completed++;
    }
    else
        time++;
}

```

```

}
printf("\n\nPID\tAT\tBT\tPR\tECT\tTAT\tWT\tRT\n");
for(i=0; i<n; i++)
{
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        i+1, at[i], bt[i], pr[i], ct[i], tat[i], wt[i], res[i]);

    avg_tat += tat[i];
    avg_wt += wt[i];
    avg_rt += res[i];
}

```

```

avg_tat /= n;
avg_wt /= n;
avg_rt /= n;
printf("\nAverage Turnaround Time = %.2f\n", avg_tat);
printf("\nAverage Waiting Time = %.2f\n", avg_wt);
printf("\nAverage Response Time = %.2f\n", avg_rt);

```

Output Code

Enter number of processes : 3
 Enter AT BT Priority for P1 : 1 4 3
 Enter AT BT Priority for P2 : 0 1 2
 Enter AT BT Priority for P3 : 2 6 1

1. Priority Non-Preemptive
2. Priority Preemptive

Enter choice : 1

Grant chart

| P2 | P1 | P3 |

PID	AT	BT	PR	CT	TAT	WT	RT
P1	1	4	3	5	4	0	0
P2	0	1	2	1	1	0	0
P3	2	6	1	11	9	3	3

Average Turnaround Time = 4.67

Average Waiting Time = 1.00

Average Response Time = 1.00

~~11.27~~ 13/3/26

Program Code (1)

(18)

For Round Robin; The program code is:

```
#include <stdio.h>
```

```
int main () {
```

```
    int n, i, t = 0, done = 0;
```

```
    int quantum;
```

```
    printf ("Enter number of processes: ");
```

```
    scanf ("%d", &n);
```

```
    int arrival[n], burst[n], remaining[n], completion[n], start[n];
```

```
    for (i = 0; i < n; i++) {
```

```
        printf ("Enter arrival time and burst time for P%d: ", i+1);
```

```
        scanf ("%d %d", &arrival[i], &burst[i]);
```

```
        remaining[i] = burst[i];
```

```
        start[i] = -1;
```

```
    }
```

```
    printf ("Enter time quantum: ");
```

```
    scanf ("%d", &quantum);
```

```
    int queue[n], front = 0, rear = 0;
```

```
    int visited[n];
```

```
    for (i = 0; i < n; i++) {
```

```
        if (arrival[i] <= t) {
```

```
            queue[rear++] = i;
```

```
            visited[i] = 1;
```

```
        }
```

```
    }
```

```
    char gantt[100][5];
```

```
    int gantt_start[100];
```

```
    int gantt_end[100];
```

```
    int gantt_count = 0;
```

```
    while (done < n) {
```

```
        if (front == rear) {
```

```
            t++;
```

```
            for (i = 0; i < n; i++) {
```

```
                if (!visited[i] && arrival[i] <= t) {
```

①. queue[rear++] = i;
visited[i] = 1;

continue;

int idx = queue[front++];
if (start[idx] == -1) start[idx] = t;
int exec = (remaining[idx] < quantum) ? remaining[idx] : quantum;
int start_exec = t;
t += exec;
remaining[idx] -= exec;

printf("ganth - process[ganth_count], \"P.%d\", idx + 1);
ganth_start[ganth_count] = start_exec;
ganth_end[ganth_count] = t;
ganth_count++;

for (i = 0; i < n; i++) {
if (!visited[i] && arrival[i] <= t) {
queue[rear++] = i;
visited[i] = 1;
}

if (remaining[idx] > 0) {
queue[rear] = idx;
} else {
~~completion[idx] = t;~~
~~done++;~~

printf("\n Ganth Chart: \n");

for (i = 0; i < ganth_count; i++) {
print(" | %s ", ganth_process[i]);

printf(" | \n");

(5)

```
for (i = 0; i < gantt_count; i++) {
```

```
    printf("%-6d", gantt_start[i]);
```

```
    printf("%d\n", gantt_end[gantt_count - 1]);
```

```
int tat[n], wt[n], rt[n];
```

```
float total_tat = 0, total_wt = 0, total_rt = 0;
```

```
for (i = 0; i < n; i++) {
```

```
    tat[i] = completion[i] - arrival[i];
```

```
    wt[i] = tat[i] - burst[i];
```

```
    rt[i] = start[i] - arrival[i];
```

```
    total_tat += tat[i];
```

```
    total_wt += wt[i];
```

```
    total_rt += rt[i];
```

```
}
```

```
printf("\n Process \t Arrival \t Burst \t Completion \t TAT \t WT \t RT \n");
```

```
for (i = 0; i < n; i++) {
```

```
    printf("P: %d \t %d \t %d \t %d \t %d \t %d \t %d \n",  
i+1, arrival[i], burst[i], completed[i], tat[i], wt[i],  
rt[i]);
```

```
}
```

```
printf("\n Average TAT: %.2f \n", total_tat / n);
```

```
printf("\n Average WT: %.2f \n", total_wt / n);
```

```
printf("\n Average RT: %.2f \n", total_rt / n);
```

```
return 0;
```

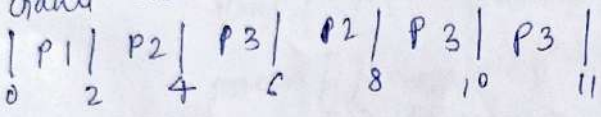
```
}
```

Output

(21)

Enter number of processes : 3
 Enter arrival time and burst time for P1 : 0 2
 Enter arrival time and burst time for P2 : 1 4
 Enter arrival time and burst time for P3 : 2 5
 Enter time quantum : 2

Gantt Chart :



Process	Arrival	Burst	Completion	TAT	WT	RT
P1	0	2	2	2	0	2
P2	1	4	8	7	3	1
P3	2	5	11	9	4	2

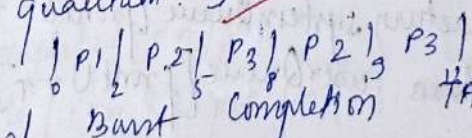
Average TAT : 6.00

Average WT : 2.33

Average RT : 1.67

Enter number of process : 3
 Enter arrival time and burst time for P1 : 0 2
 Enter arrival time and burst time for P2 : 1 4
 Enter arrival time and burst time for P3 : 2 5
 Enter time quantum : 3

Gantt chart :



Process	Arrival	Burst	Completion	TAT	WT	RT
P1	0	2	2	2	0	2
P2	1	4	9	8	4	1
P3	2	5	11	9	4	3

Average TAT : 6.33

Average WT : 2.17

Average RT : 2.00

A smaller time quantum in Round Robin gives better response for short process but more context switching, while a larger quantum reduces switching and behaves like FCFS, potentially increasing waiting for short process.

Program Code (2)

(22)

Multilevel Queue Scheduling algorithm:

```
#include <stdio.h>
#include <string.h>
#define MAX 100

type def struct {
    int pid;
    int arrival;
    int burst;
    int remaining;
    int completion;
    char type[10];
}

Process;
Process process[MAX];
int systemQueue[MAX], userQueue[MAX];
int frontS = 0, rearS = -1;
int frontU = 0, rearU = -1;
int gantt[MAX];
int gIndex = 0;
void enqueueSystem (int index) { systemQueue[++rearS] = index; }
void enqueueUser (int index) { userQueue[++rearU] = index; }
int dequeueSystem () { return systemQueue[frontS++]; }
int dequeueUser () { return userQueue[frontU++]; }
int isSystemEmpty () { return frontS > rearS; }
int isUserEmpty () { return frontU > rearU; }
int main () {
    int n;
    printf ("Enter number of processes.");
    scanf ("%d", &n);
    for (int i=0; i<n; i++) {
        printf ("\nEnter Process ID.");
        scanf ("%d", &process[i].pid);
    }
}
```

```

    printf("Arrival time: ");
    scanf("%f", &process[i].arrival);
    printf("Burst time: ");
    scanf("%f", &process[i].burst);
    printf("Type (System/User): ");
    scanf("%s", process[i].type);
    process[i].remaining = process[i].burst;
    process[i].start = 0;
}

for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - 1; j++) {
        if (process[j].arrival > process[j+1].arrival) {
            process temp = process[j];
            process[j] = process[j+1];
            process[j+1] = temp;
        }
    }
}

int current_time = 0, completed = 0, i = 0;
int current_process = -1;
while (completed < n) {
    while (i < n && process[i].arrival <= current_time) {
        if (strcmp(process[i].type, "System") == 0) {
            enqueueSystem(i);
        } else {
            enqueueUser(i);
        }
        i++;
    }

    if (current_process != -1) {
        if (strcmp(process[current_process].type, "User") == 0
            && !isSystemEmpty()) {

```

(29)

```
enqueue User (current-proc);  
current-proc = -1;
```

```
}  
if (current-proc == -1) {  
    if (!isSystemEmpty()) {  
        current-proc = dequeueSystem();  
    } else if (!isUserEmpty()) {  
        current-proc = dequeueUser();  
    } else {  
        gantt[gIndex++] = -1;  
        current-time++;  
        continue;  
    }  
}
```

```
{  
if (procen[current-proc].started == 0) {  
    procen[current-proc].response =  
        current-time - procen[current-proc].arrival;  
    procen[current-proc].started = 1;
```

```
{  
procen[current-proc].remaining--;  
gantt[gIndex++] = procen[current-proc].pid;  
current-time++;
```

```
if (procen[current-proc].remaining == 0) {  
    procen[current-proc].completion = current-time;  
    procen[current-proc].turnaround = procen[current-proc].  
        completion - procen[current-proc].arrival;  
    procen[current-proc].waiting = procen[current-proc].  
        turnaround - procen[current-proc].pid, current-time;  
    printf ("process %d completed at time %d\n",  
        procen[current-proc].pid, current-time,  
        completed++);  
    current-proc = -1;
```

(25)

```

printf ("\n Gantt Chart: \n /");
for (int i = 0; i < gIndex; i++) {
    if (gantt[i] == -1) {
        printf ("IDLE |");
    } else {
        printf ("P.%d |", gantt[i]);
    }
}

```

```

printf ("\n 0");
for (int i = 1; i <= gIndex; i++) {
    printf (" %d", i);
}

```

```

printf ("\n");

```

```

float totalTAT = 0, totalWT = 0, totalRT = 0;

```

```

printf ("\n Proccen Details: \n");

```

```

printf ("PID \t AT \t BT \t CT \t TAT \t WT \t RT \n");

```

```

for (int i = 0; i < n; i++) {
    printf ("%d \t %d \t %d \t %d \t %d \t %d \n",
        procen[i].pid,
        procen[i].arrival, procen[i].burst,
        procen[i].completion, procen[i].turnaround,
        procen[i].waiting,
        procen[i].response);
}

```

```

totalTAT += procen[i].turnaround;

```

```

totalWT += procen[i].waiting;

```

```

totalRT += procen[i].response;

```

```

}
printf ("\n Average Turnaround Time = %.2f", totalTAT/n);
printf ("\n Average Waiting Time = %.2f", totalWT/n);
printf ("\n Average Response Time = %.2f", totalRT/n);
return 0;
}

```

Output

(k)

Enter number of processes : 3

Enter process ID : 3

Arrival time : 0

Burst time : 3

Type (System/User) : system

Enter process ID : 2

Arrival Time : 2

Burst Time : 2

Type (System/User) : User

Enter process ID : 3

Arrival Time : 3

Burst Time : 4

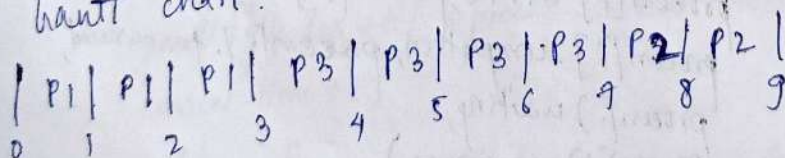
Type (System/User) : System

Process 1 completed at time 3

Process 3 completed at time 7

Process 2 completed at time 9

Gantt chart:



Process Details:

PID	AT	BT	CT	TAT	WT	RT
1	0	3	3	3	0	0
2	2	2	9	7	5	5
3	3	4	7	4	0	0

Average Turnaround Time : 4.67

Average Waiting Time = 1.67

Average Response Time = 1.67

Comparison

<u>Feature</u>	<u>Multilevel Queue</u>	<u>Round Robin</u>
Priority	(System > user)	No Priority, equal share
Preemption	User can be preempted by system.	Preempted only by Time Quantum.
Fairness	Queue-specific, may starve user	Fair across all processes
Best use case	OS-critical task, real-time system	Time sharing
Complexity	moderate (Queue Management)	Simple
Flexibility	Fixed queue priorities	Quantum adjustable for responsiveness.

Q2 21/3/26

Q. Write a C program to stimulate Real-time CPU scheduling algorithms.

- rate monotonic
- Earliest deadline first
- Proportional scheduling

Program Code

```
#include <stdio.h>
#include <math.h>
#define MAX 10
typedef struct {
    int id;
    int burst;
    int deadline;
    int period;
    int weight;
    int ct, wt, tat;
} process;

void sortByDeadline (Process p[], int n) {
    for (int i=0; i<n-1; i++) {
        for (int j=i+1; j<n; j++) {
            if (p[i].deadline > p[j].deadline) {
                process temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}

void EDF (Process p[], int n) {
    process temp[MAX];
    for (int i=0; i<n; i++) temp[i] = p[i];
    sortByDeadline (temp, n);
    float util = 0;
    for (int i=0; i<n; i++) {
        util += (float)temp[i].burst / temp[i].deadline;
    }
}
```

```

printf ("|n == Earliest deadline first (EDF) scheduling == |n");
printf ("CPU utilization: %d.2f |n", util);
if (util <= 1.0) {
    printf ("Schedulable (utilization <= 1) |n");
} else {
    printf ("Schedulable & utilization is not possible |n");
}

```

```

int time = 0;
printf ("ID BF deadline CT WT TAT |n");
for (int i=0; i<n; i++) {
    time += temp[i].burst;
    temp[i].ct = time;
    temp[i].tat = temp[i].ct;
    temp[i].wt = temp[i].tat - temp[i].burst;
    printf ("%d %d %d %d %d %d |n",
        temp[i].id, temp[i].burst, temp[i].deadline,
        temp[i].ct, temp[i].tat);
}

```

```

void RMS (process p[], int n) {
    process temp[MA];
    for (int i=0; i<n; i++) temp[i] = p[i];
    sortByPeriod (temp, n);
    float util = 0;
    float util = 0;
    for (int i=0; i<n; i++) {
        util += (float) temp[i].burst / temp[i].period;
    }
    float ambound = m * (pow(2.0, 1.0/n) - 1);
    printf ("|n == Rate Monotonic scheduling (RMS) == |n");
    printf ("CPU utilization %d.2f |n", util);
}

```

Number of processes: 3

Enter process details:

Process 0:

Burst Time: 1

Deadline (for EDF): 3

Period (for RMS): 4

Weight (for Proportional): 2

Process 1:

Burst Time: 1

Deadline (for EDF): 2

Period (for RMS): 4

Weight (for proportional): 3

Process 2:

Burst Time: 5

Deadline (for EDF): 4

Period (for RMS): 2

Weight (for proportional): 1

=== Earliest Deadline First (EDF) Scheduling ===

CPU Utilization: 2.08

Not schedulable

ID	CT	Deadline	WT	TAT
1	1	2	1	0
0	1	3	2	1
2	5	4	7	2

=== Rate Monotonic Scheduling (RMS) ===

CPU Utilization: 3.00

RM Bound: 0.7998

May Not Be Schedulable

ID	BF	Period	CT	WT	TAT
2	5	2	5	0	5
1	1	4	6	5	6
0	1	4	7	6	7

=== Proportional scheduling ===

ID	Weight	CPU share (%)
0	2	33.33%
1	3	50.00%
2	1	16.67%

Q4 id4/26

LAB-5

1) Bounded Buffer

Program code:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE
int buffer[BUFFER_SIZE];
int in = 0, out = 0;
sem_t empty, full;
pthread_mutex_t mutex;

void* producer(void* arg) {
    int item;
    for (int i = 0; i < 7; i++) {
        item = i;
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);
        buffer[in] = item;
        printf("Produced: %d at buffer[%d]\n", item, in);
        in = (in + 1) % BUFFER_SIZE;
```

```
pthread_mutex_unlock(&mutex);  
sem_post(&full);  
sleep(1);  
}
```

```
void consumer(void *arg){  
    int item;  
    for(int i = 0; i < 7; i++){  
        sem_wait(&full);  
        pthread_mutex_lock(&mutex);  
        item = buffer[out];  
        printf("Consumed : %d from buffer [%d]\n", item, out);  
        out = (out + 1) % Buffer-Size;  
        pthread_mutex_unlock(&mutex);  
        sem_post(&empty);  
        sleep(2);  
    }  
}
```

```
int main() {  
    pthread_t p, c;  
    sem_init(&empty, 0, Buffer-Size);  
    sem_init(&full, 0, 0);  
    pthread_mutex_init(&mutex, NULL);  
    pthread_create(&p, NULL, producer, NULL);  
    pthread_create(&c, NULL, consumer, NULL);  
    pthread_join(p, NULL);  
    pthread_join(c, NULL);  
    sem_destroy(&empty);  
    sem_destroy(&full);  
    pthread_mutex_destroy(&mutex);  
    return 0;  
}
```

O/P

2) Produced: 0 at buffer[0]
Consumed: 0 at buffer[0]
Produced: 1 at buffer[1]
Consumed: 1 at buffer[1]
Produced: 2 at buffer[2]
Produced: 3 at buffer[3]
Consumed: 2 from buffer[2]
Produced: 4 at buffer[4]
Produced: 5 at buffer[0]
Consumed: 3 from buffer[3]
Produced: 6 at buffer[1]
Consumed: 4 at buffer[4]
Consumed: 5 at buffer[0]
~~Consumed: 6 at buffer[1]~~

23

24

25

26

27

28

29

30

31

32

33

34

35

36

37

38

39

40

41

42

43

44

45

46

47

48

49

50

51

52

53

54

55

56

57

58

59

60

61

62

63

64

65

66

67

68

2) Dining Philosopher

06

Program code:

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5
sem_t mutex;
sem_t S[N];

int state[N];

int LEFT(int i) {
    return (i + N - 1) % N;
}

int RIGHT(int i) {
    return (i + 1) % N;
}

void test(int i) {
    if (state[i] == 1 &&
        state[LEFT(i)] != 2 &&
        state[RIGHT(i)] != 2) {
        state[i] = 2;
        printf("Philosopher %d is eating\n", i);
        sem_post(&S[i]);
    }
}

void take_forks(int i) {
    sem_wait(&mutex);
    state[i] = 1;
    printf("Philosopher %d is hungry\n", i);
    test(i);
    sem_post(&mutex);
    sem_wait(&S[i]);
}
```

```
void put_forks(int i) {  
    sem_wait(&mutex);  
    state[i] = 0;  
    printf("Philosopher %d is thinking\n", i);  
    test(LEFT(i));  
    test(RIGHT(i));  
    sem_post(&mutex);  
}
```

```
void * philosopher(void * num) {  
    int i = *(int *) num;  
    while(1) {  
        printf("Philosopher %d is thinking\n", i);  
        sleep(1);  
        take_forks(i);  
        sleep(2);  
        put_forks(i);  
    }  
}
```

```
int main() {  
    pthread_t thread[N];  
    int phil[N];  
    sem_init(&mutex, 0, 1);  
    for (int i = 0; i < N; i++)  
        sem_init(&S[i], 0, 0);  
    for (int i = 0; i < N; i++) {  
        phil[i] = i;  
        pthread_create(&thread[i], NULL, philosopher,  
            &phil[i]);  
    }  
    for (int i = 0; i < N; i++)  
        pthread_join(thread[i], NULL);  
    return 0;  
}
```

O/P

philosopher	0	is	thinking
philosopher	1	is	thinking
philosopher	2	is	thinking
philosopher	3	is	Thinking
philosopher	4	is	Thinking
philosopher	0	is	hungry
philosopher	0	is	eating
philosopher	0	is	hungry
philosopher	2	is	eating
philosopher	2	is	hungry
philosopher	1	is	hungry
philosopher	3	is	thinking
philosopher	0	is	eating
philosopher	1	is	hungry
philosopher	4	is	thinking
philosopher	2	is	eating
philosopher	3	is	thinking
philosopher	1	is	hungry
philosopher	0	is	eating
philosopher	0	is	thinking
philosopher	3	is	eating
philosopher	4	is	eating

#include <stdio.h>

```

int main() {
    int n, m, i, j, k;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter number of resources: ");
    scanf("%d", &m);
    int allocation[n][m], max[n][m], need[n][m];
    int available[m];
    printf("\nEnter Allocation Matrix: \n");
    for(i=0; i<n; i++) {
        for(j=0; j<m; j++) {
            scanf("%d", &allocation[i][j]);
        }
    }
    printf("\nEnter Maximum Demand Matrix: \n");
    for(i=0; i<n; i++) {
        for(j=0; j<m; j++) {
            scanf("%d", &max[i][j]);
        }
    }
    printf("\nEnter Available Resources: \n");
    for(i=0; i<m; i++) {
        scanf("%d", &available[i]);
    }
    for(i=0; i<n; i++) {
        for(j=0; j<m; j++) {
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }
    int finish[n], safeSequence[n];
    int work[m];
    for(i=0; i<n; i++) {
        finish[i] = 0;
    }
    for(i=0; i<m; i++) {
        work[i] = available[i];
    }
    int count = 0;
    while(count < n) {
        int found = 0;
        for(i=0; i<n; i++) {
            if(finish[i] == 0) {
                for(j=0; j<m; j++) {
                    if(need[i][j] > work[j])
                        break;
                }
            }
        }
    }
}

```

```

if (j == m) {
    for (k = 0; k < m; k++) {
        work[k] += allocation[i][k];
    }
    safeSequence[count++] = i;
    finish[i] = 1;
    found = 1;
}
}

```

```

if (found == 0)
    break;

```

```

if (count == n) {
    printf("\nSystem is in a safe state.\n");
    printf("Safe sequence is: ");
    for (i = 0; i < n; i++) {
        printf("P%d", safeSequence[i]);
        if (i == n - 1)
            printf(" -> ");
    }
}

```

```

} else {
    printf("\n");
}

```

```

printf("\nSystem is not in safe state.\n");

```

```

printf("Safe sequence is: ");

```

```

for (i = 0; i < n; i++) {

```

```

    printf("P%d", safeSequence[i]);

```

```

}

```

```

return 0;

```

I

Output

(11)

Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:

0	1	0
2	0	0
3	0	2
2	1	1
0	0	2

Enter Maximum Demand Matrix:

7	5	3
3	2	2
9	0	2
2	2	2
4	3	3

Enter Available Resources:

3	3	2
---	---	---

System is in a safe state.

Safe sequence is: $P1 \rightarrow P3 \rightarrow P4 \rightarrow P0 \rightarrow P2$

Q15/26

DEADLOCK DETECTION

42

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, m, i, j, k;
```

```
    printf("Enter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("Enter number of resources: ");
```

```
    scanf("%d", &m);
```

```
    int allocation[n][m], request[n][m];
```

```
    int available[m];
```

```
    printf("\nEnter Allocation Matrix: \n");
```

```
    for (i = 0; i < n; i++) {
```

```
        for (j = 0; j < m; j++) {
```

```
            scanf("%d", &allocation[i][j]);
```

```
        }
```

```
    }
```

```
    printf("\nEnter Request Matrix: \n");
```

```
    for (i = 0; i < n; i++) {
```

```
        for (j = 0; j < m; j++) {
```

```
            scanf("%d", &request[i][j]);
```

```
        }
```

```
    }
```

```
    printf("\nEnter Available Resources: \n");
```

```
    for (i = 0; i < m; i++) {
```

```
        scanf("%d", &available[i]);
```

```
    } int finish[n];
```

```
    int work[m];
```

```
    for (i = 0; i < n; i++) {
```

```
        int sum = 0;
```

```
        for (j = 0; j < m; j++) {
```

```
            sum += allocation[i][j];
```

```
        } if (sum == 0)
```

```
            finish[i] = 1;
```

```
        else
```

```
            finish[i] = 0;
```

```
    }
```

(40)

```
for (i=0; i<m; i++) {  
    work[i] = available[i];
```

```
int found;
```

```
do {
```

```
    found = 0;
```

```
    for (i=0; i<n; i++) {
```

```
        if (finish[i] == 0) {
```

```
            for (j=0; j<m; j++) {
```

```
                if (request[i][j] > work[j]) {  
                    break;
```

```
            }  
        }
```

```
    } if (j == m) {
```

```
        for (k=0; k<m; k++) {
```

```
            work[k] += allocation[i][k];
```

```
        }
```

```
        finish[i] = 1;
```

```
        found = 1;
```

```
    }
```

```
}
```

```
{
```

```
while (found);
```

```
int deadlock = 0;
```

```
printf("\n Processes in Deadlock: \n");
```

```
for (i=0; i<n; i++) {
```

```
    if (finish[i] == 0) {
```

```
        printf("P/d", i);
```

```
        deadlock = 1;
```

```
    }
```

```
}
```

```
if (deadlock == 0) {
```

```
    printf("No deadlock detected.
```

```
    All processes can finish. \n");
```

```
} else {
```

printf("\n Deadlock detected.\n");

} return 0;

}

O/P

Enter ~~data~~ number of processes: 5

Enter number of resources: 3

Enter Allocation Matrix:

0 1 0

2 0 0

3 0 3

2 1 1

0 0 2

Enter Request Matrix:

0 0 0

2 0 2

0 0 0

1 0 1

0 0 2

Enter Available Resources:

0 0 0

~~Process in Deadlock:~~

No deadlock detected. All processes can finish.

~~Ans~~

Program code

#include <stdio.h>

void firstFit (int blocks[7], int m, int process[7], int n) {

int allocation[n];

int used[m];

for (int i = 0; i < n; i++)

allocation[i] = -1;

for (int i = 0; i < m; i++)

used[i] = 0;

for (int i = 0; i < n; i++) {

for (int j = 0; j < m; j++) {

if (blocks[j] >= process[i] && used[j] == 0) {

allocation[i] = j;

used[j] = 1;

break;

}

}

}

printf("\n --- First Fit --- \n");

printf("Process No. \t Process Size \t Block No. \n");

for (int i = 0; i < n; i++) {

printf("%d \t %d \t %d", i+1, process[i],

if (blocks[j] >= allocation[i] == -1)

printf("%d \n", allocation[i] + 1);

else printf("Not Allocated \n");

}

void BestFit (int blocks[7], int m, int process[7], int n) {

int allocation[n];

int used[m];

for (int i = 0; i < n; i++)

allocation[i] = -1;

for (int i = 0; i < m; i++)

used[i] = 0;

for (int i = 0; i < n; i++) {

int best = -1;

```
for(int j=0; j<m; j++){
    if (blocks[j] >= process[i] & used[j]==0){
        if (best == -1 || blocks[j] < blocks[best])
            best = j;
    }
}
```

```
if (best != -1){
    allocation[i] = best;
    used[best] = 1;
}
```

```
}
printf("\n--- Best fit ---\n");
printf("Process No. \t Process size \t Block No.\n");
```

```
for(int i=0; i<n; i++){
    printf("%d\t\t\t\t\t", i+1, process[i]);
    if (allocation[i] != -1)
        printf("%d\n", allocation[i]+1);
    else
        printf("Not allocated\n");
}
```

```
void Worst Fit (int blocks[], int m, int process[], int n){
    int allocation[n];
    int used[m];
```

```
for(int i=0; i<n; i++){
    allocation[i] = -1;
}
```

```
for(int i=0; i<m; i++){
    used[i] = 0;
}
```

```
for(int i=0; i<n; i++){
    int worst = -1;
```

```
for(int j=0; j<m; j++){
    if (blocks[j] >= process[i] &
        used[j]==0){
```

if (worst == -1 || blocks[j] > blocks[worst]) (13)

worst = j;

```
if (worst != -1) {  
    allocation[i] = worst;  
    used[worst] = 1;  
}
```

```
printf("\n --- Worst Fit --- \n");  
printf("Process No. \t Process Size \t Block No. \n");  
for (int i = 0; i < n; i++) {  
    printf("%d \t %d \t", i+1, process[i]);  
    if (allocation[i] != -1)  
        printf("%d \n", allocation[i] + 1);  
    else  
        printf("Not allocated \n");  
}
```

```
int main() {  
    int m, n;  
    printf("Enter the number of memory blocks: ");  
    scanf("%d", &m);  
    int blocks[m];  
    printf("Enter sizes of memory blocks: \n", m);  
    for (int i = 0; i < m; i++)  
        scanf("%d", &blocks[i]);  
    printf("\n Enter number of processes: ");  
    scanf("%d", &n);  
    int process[n];  
    printf("Enter sizes of processes: \n", n);  
    for (int i = 0; i < n; i++)  
        scanf("%d", &process[i]);  
    firstFit(blocks, m, process, n);  
    bestFit(blocks, m, process, n);  
    worstFit(blocks, m, process, n);  
    return 0;  
}
```

Trailing

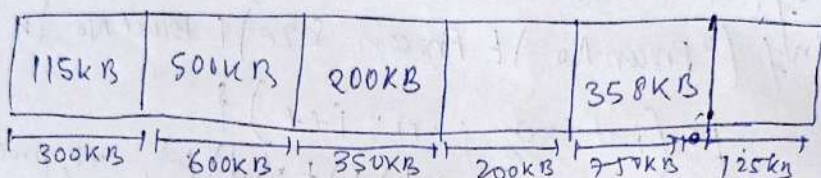
48

Given 6 memory partitions:

300KB, 600KB, 350KB, 200KB, 750KB, 125KB

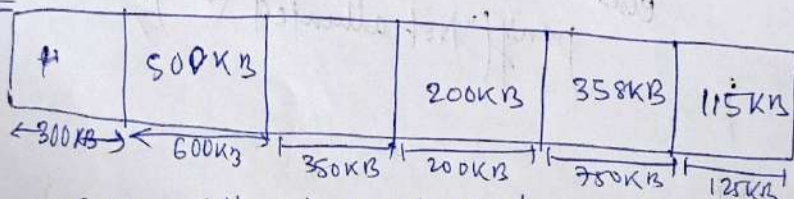
order (115KB, 500KB, 358KB, 200KB, 375KB)

First fit



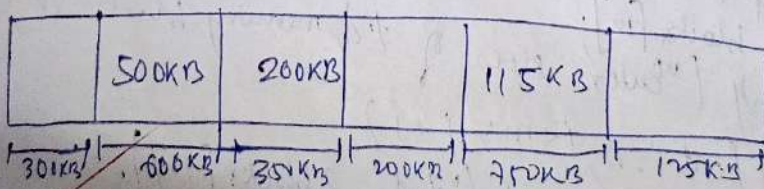
375KB will not be allocated anywhere in the memory allocated block.

Best fit



375KB will not be allocated for the best fit.

Worst fit



In worst fit both 358KB and 375KB will not be allocated.

O/P code

Enter number of memory blocks: 6

Enter size of 6 memory blocks.

300 600 350 200 750 125

Enter number of processes: 5

Enter sizes of 5 processes:

115 500 358 200 375

--- First Fit ---

Process No.	Process Size	Block No
1	115	1
2	300	2
3	358	5
4	200	3
5	375	Not allocated

--- Best Fit ---

Process No.	Process Size	Block No
1	115	6
2	300	2
3	358	5
4	200	4
5	375	Not allocated

--- Worst Fit ---

Process No	Process size	Block No
1	115	5
2	300	2
3	358	Not Allocated
4	200	3
5	375	Not Allocated

Ans 15/20

Program Code

LAB # Page faults

60

```
void fifo(int page[], int n, int frames){
    int frame[10], i, j;
    int k = 0, fault = 0, found;
    for(i = 0; i < frames; i++){
        frame[i] = -1;
    }
    printf("\n == FIFO page Replacement == \n");
    for(i = 0; i < n; i++){
        found = 0;
        for(j = 0; j < frames; j++){
            if(frame[j] == page[i]){
                found = 1;
                break;
            }
        }
        if(found == 0){
            frame[k] = page[i];
            k = (k + 1) % frames;
            fault++;
        }
        printf("\n %d -> ", page[i]);
        for(j = 0; j < frames; j++){
            if(frame[j] != -1)
                printf("%d ", frame[j]);
            else
                printf("- ");
        }
        printf("\n");
    }
    printf("\n\n Total FIFO page faults = %d \n", fault);
}
```

```
void lru(int pages[], int n, int frames){
    int frame[10], time[10];
    int i, j, pos = 0;
    int fault = 0, found, min;
    int counter = 0;
    for(i = 0; i < frames; i++){
        frame[i] = -1;
        time[i] = 0;
    }
}
```

(51)

```

printf("\n --- LRU Page Replacement --- \n");
for(i=0; i<n; i++){
    found = 0;
    for(j=0; j<frames; j++){
        if(frame[j] == pages[i]){
            counter++;
            time[j] = counter;
            found = 1;
            break;
        }
    }
    if(found == 0){
        min = time[0];
        pos = 0;
        for(j=1; j<frames; j++){
            if(time[j] < min){
                min = time[j];
                pos = j;
            }
        }
        counter++;
        frames[pos] = pages[i];
        time[pos] = counter;
        fault++;
    }
    printf("\n i: %d -> ", pages[i]);
    for(j=0; j<frames; j++){
        if(frame[j] != -1){
            printf("%d ", frame[j]);
        }
        else printf("- ");
    }
    printf("\n");
}
printf("\n Total LRU pages faults = %d \n", fault);

```

void optimal (int pages[], int n, int page-frames) {

int frame[10];

int i, j, k;

int pos = -1, fault = 0, found;

int farthest, index;

for (i = 0; i < pages; i++)

frame[i] = -1;

printf("In-Optimal Page Replacement - \n");

for (i = 0; i < n; i++) {

found = 0;

for (j = 0; j < frames; j++) {

if (frame[j] == pages[i]) {

found = 1;

break;

}

}

if (found == 0) {

pos = -1;

farthest = -1;

for (j = 0; j < frames; j++) {

index = -1;

for (k = i + 1; k < n; k++) {

if (frame[j] ==

pages[k]) {

index = k;

break;

}

if (index == -1) {

pos = j;

break;

if (index > farthest) {

farthest = index;

pos = j;

frame[par] = pages[i];

fault++;

(53)

printf("\n %d → ", pages[i]);

for(j=0; j<frames; j++) {

if (frame[j] == -1) {

printf(" %d ", frame[j]);

} else {

printf(" - ");

printf("\n Total Optimal Page faults = %d", fault);

}

int main () {

int pages[50];

int n, frames, i;

printf("Enter number of pages: ");

scanf("%d", &n);

printf("Enter page reference string: \n");

for (i=0; i<n; i++) {

scanf("%d", &pages[i]);

}

printf("Enter number of frames: ");

scanf("%d", &frames);

fifo(pages, n, frames);

lru(pages, n, frames);

optimal(pages, n, frames);

return 0;

}

O/P

54

Enter number of pages: 12

Enter page reference string:

7 0 18 46 8 7 7 0 0 2

Enter number of frames: 3

--- FIFO Page Replacement ---

7 → 7 - -

0 → 7 0 -

1 → 7 0 1

8 → 8 0 1

4 → 8 4 1

6 → 8 4 6

5 → 5 4 6

7 → 5 7 6

7 → 5 7 6

0 → 5 7 0

0 → 5 7 0

2 → 2 7 0

Total FIFO page faults = 10

--- LRU Page Replacement ---

7 → 7 - -

0 → 7 0 -

1 → 7 0 1

8 → 8 0 1

4 → 8 4 1

6 → 8 4 1

5 → 5 4 6

7 → 5 7 6

7 → 5 7 6

0 → 5 7 0

0 → 5 7 0

2 → 2 7 0

Total LRU Page faults = 10

--- Optimal Page Replacement ---

7 → 7 - -

0 → 7 0 -

1 → 7 0 1

8 → 7 0 8

4 → 7 0 4

6 → 7 0 6

5 → 7 0 5

7 → 7 0 5

7 → 7 0 5

0 → 7 0 5

0 → 7 0 5

2 → 2 0 5

Total, optimal Page faults = 8

888
22/5/26